3D Spaces Modelling

Lecture 7

Supervisor - Prof. Dr. Mohammed Aquil Mirza, COMP
Student Mentors - Mr. Ruan Haozhe, Mr. Jiang Suyu, Mr. Zhao Jiuyuan

June 2026
The Hong Kong Polytechnic University

Why 3D for HCR

The scene is 3D.
The scalp is a curved surface, the tool has a pose, and the robot arm moves through space. We need a language for direction, distance, and motion.

Vectors come first.
Before we model a head or plan a cut path, we need vectors and the basic operations: addition, dot, and cross.

Then coordinate systems.
The same point in space can be described in Cartesian, cylindrical, or spherical coordinates. Pick the one that matches the geometry.

1 Vectors

Scalar and vector quantities

Scalar quantities.
Have magnitude only (e.g., mass, temperature, time, distance).

Vector quantities.
Have both magnitude and direction (e.g., displacement, velocity, acceleration, force).

Notation.
Vectors are represented with arrows $\vec{v}$, while scalars use regular type s.

Components of a vector

Decomposition.
Any vector $\vec{A}$ can be decomposed into components along coordinate axes.

Rectangular (Cartesian) components.

$$\vec{A} = A_x\hat{i} + A_y\hat{j} + A_z\hat{k}$$

where A_x, A_y, A_z are scalar components and $\hat{i}$, $\hat{j}$, $\hat{k}$ are unit vectors.

Magnitude and parallel vectors

Magnitude.

$$|\vec{A}| = \sqrt{A_x^2 + A_y^2 + A_z^2}$$

Parallel vectors.
Two vectors $\vec{A}$ and $\vec{B}$ are parallel if $\vec{A} = \lambda \vec{B}$ where λ is a scalar.

Vector addition and subtraction

Component-wise.
Vectors are added or subtracted component-wise:

$$\vec{A} + \vec{B} = (A_x + B_x)\hat{i} + (A_y + B_y)\hat{j} + (A_z + B_z)\hat{k}$$

$$\vec{A} - \vec{B} = (A_x - B_x)\hat{i} + (A_y - B_y)\hat{j} + (A_z - B_z)\hat{k}$$

Properties.
Commutative: $\vec{A} + \vec{B} = \vec{B} + \vec{A}$. Scalar multiplication: $k(\vec{A} + \vec{B}) = k\vec{A} + k\vec{B}$.

Vector dot product

Definition.
Vector dot product means a projection of one vector on another. The product is a scalar.

$$\vec{A} \cdot \vec{B} \equiv |\vec{A}||\vec{B}| \cos\theta$$

Properties.
Commutative: $\vec{A} \cdot \vec{B} = \vec{B} \cdot \vec{A}$. Self dot product: $\vec{A} \cdot \vec{A} = |\vec{A}|^2$. Distributive: $\vec{A} \cdot (\vec{B} + \vec{C}) = \vec{A} \cdot \vec{B} + \vec{A} \cdot \vec{C}$.

Dot product in components

Orthonormal basis.
The standard basis vectors $\{\hat{i}, \hat{j}, \hat{k}\}$ satisfy $\hat{e}_i \cdot \hat{e}_j = \delta_{ij}$.

Component formula.
For $\vec{A} = A_x\hat{i} + A_y\hat{j} + A_z\hat{k}$ and $\vec{B} = B_x\hat{i} + B_y\hat{j} + B_z\hat{k}$:

$$\vec{A} \cdot \vec{B} = A_xB_x + A_yB_y + A_zB_z$$

Angle between vectors.

$$\cos\theta = \frac{A_xB_x + A_yB_y + A_zB_z}{\sqrt{A_x^2 + A_y^2 + A_z^2}\sqrt{B_x^2 + B_y^2 + B_z^2}}$$

Vector cross product

Definition.
The cross product produces a vector perpendicular to both input vectors. The result is a vector.

$$\vec{A} \times \vec{B} = |\vec{A}||\vec{B}| \sin\theta \, \hat{n}$$

where $\hat{n}$ is the unit vector perpendicular to both $\vec{A}$ and $\vec{B}$ (direction given by right-hand rule).

Properties.
Anti-commutative: $\vec{A} \times \vec{B} = -\vec{B} \times \vec{A}$. Self cross product: $\vec{A} \times \vec{A} = \vec{0}$. Distributive: $\vec{A} \times (\vec{B} + \vec{C}) = \vec{A} \times \vec{B} + \vec{A} \times \vec{C}$.

Cross product of basis vectors

Orthonormal basis cross products.

$$\hat{i} \times \hat{j} = \hat{k}, \quad \hat{j} \times \hat{k} = \hat{i}, \quad \hat{k} \times \hat{i} = \hat{j}$$
$$\hat{j} \times \hat{i} = -\hat{k}, \quad \hat{k} \times \hat{j} = -\hat{i}, \quad \hat{i} \times \hat{k} = -\hat{j}$$
$$\hat{i} \times \hat{i} = \hat{j} \times \hat{j} = \hat{k} \times \hat{k} = \vec{0}$$

Cross product in components

Component formula.
For $\vec{A} = A_x\hat{i} + A_y\hat{j} + A_z\hat{k}$ and $\vec{B} = B_x\hat{i} + B_y\hat{j} + B_z\hat{k}$:

$$\vec{A} \times \vec{B} = (A_yB_z - A_zB_y)\hat{i} + (A_zB_x - A_xB_z)\hat{j} + (A_xB_y - A_yB_x)\hat{k}$$

Determinant form.

$$\vec{A} \times \vec{B} = \begin{vmatrix} \hat{i} & \hat{j} & \hat{k} \\ A_x & A_y & A_z \\ B_x & B_y & B_z \end{vmatrix}$$

Geometric properties.
$|\vec{A} \times \vec{B}| = |\vec{A}||\vec{B}| \sin\theta$ represents the area of the parallelogram formed by $\vec{A}$ and $\vec{B}$. If $\vec{A} \times \vec{B} = \vec{0}$, then $\vec{A}$ and $\vec{B}$ are parallel.

2 Coordinate Systems and Different Unit Vectors

Unit vectors

Definition.
A unit vector $\hat{e}$ is a vector with magnitude 1.

Decomposition.
Any vector $\vec{A}$ can be expressed as

$$\vec{A} = |\vec{A}|\hat{e}_A \quad \text{where} \quad \hat{e}_A = \frac{\vec{A}}{|\vec{A}|}$$

Cartesian (Rectangular) coordinates

Point.
(x, y) or (x, y, z) where $x, y, z \in \mathbb{R}$.

2D.
Point (x, y) with position vector $\vec{r} = x\hat{e}_x + y\hat{e}_y$.

3D.
Point (x, y, z) with position vector $\vec{r} = x\hat{e}_x + y\hat{e}_y + z\hat{e}_z$.

Unit vectors.
$\hat{e}_x = \hat{i}$, $\hat{e}_y = \hat{j}$, $\hat{e}_z = \hat{k}$ (constant directions).

Polar coordinates (2D)

Point.
(r, θ) where $r \geq 0$, $0 \leq \theta < 2\pi$.

Conversion.
$x = r\cos\theta, \quad y = r\sin\theta, \quad r = \sqrt{x^2 + y^2}, \quad \theta = \arctan(y/x).$

Unit vectors.
$\hat{e}_r$ (radial), $\hat{e}_\theta$ (tangential).

Position vector.
$\vec{r} = r\hat{e}_r = x\hat{e}_x + y\hat{e}_y$.

Polar unit vector relations

Polar in Cartesian.

$$\hat{e}_r = \cos\theta\,\hat{e}_x + \sin\theta\,\hat{e}_y$$

$$\hat{e}_\theta = -\sin\theta\,\hat{e}_x + \cos\theta\,\hat{e}_y$$

Inverse.

$$\hat{e}_x = \cos\theta\,\hat{e}_r - \sin\theta\,\hat{e}_\theta$$

$$\hat{e}_y = \sin\theta\,\hat{e}_r + \cos\theta\,\hat{e}_\theta$$

Cylindrical coordinates (3D)

Point.
(ρ, ϕ, z) where $\rho \geq 0$, $0 \leq \phi < 2\pi$.

Conversion.
$x = \rho \cos\phi, \quad y = \rho \sin\phi, \quad z = z.$

Unit vectors.
$\hat{e}_\rho$ (radial), $\hat{e}_\phi$ (azimuthal), $\hat{e}_z$ (vertical).

Position vector.
$\vec{r} = \rho\hat{e}_\rho + z\hat{e}_z = x\hat{e}_x + y\hat{e}_y + z\hat{e}_z.$

Cylindrical unit vector relations

Cylindrical in Cartesian.

$$\hat{e}_\rho = \cos\phi\,\hat{e}_x + \sin\phi\,\hat{e}_y$$
$$\hat{e}_\phi = -\sin\phi\,\hat{e}_x + \cos\phi\,\hat{e}_y$$
$$\hat{e}_z = \hat{e}_z$$

Inverse.

$$\hat{e}_x = \cos\phi\,\hat{e}_\rho - \sin\phi\,\hat{e}_\phi$$
$$\hat{e}_y = \sin\phi\,\hat{e}_\rho + \cos\phi\,\hat{e}_\phi$$
$$\hat{e}_z = \hat{e}_z$$

Spherical coordinates (3D)

Point.
(r, θ, ϕ) where $r \geq 0$, $0 \leq \theta \leq \pi$, $0 \leq \phi < 2\pi$. We follow the physics convention: θ is the polar angle (from $+z$) and ϕ is the azimuthal angle (around z); the mathematics convention swaps the two symbols, so always check which is which before copying a formula.

Conversion.
$x = r \sin\theta \cos\phi, \quad y = r \sin\theta \sin\phi, \quad z = r \cos\theta.$

Unit vectors.
$\hat{e}_r$ (radial), $\hat{e}_\theta$ (polar angle), $\hat{e}_\phi$ (azimuthal).

Position vector.
$\vec{r} = r\hat{e}_r = x\hat{e}_x + y\hat{e}_y + z\hat{e}_z.$

Spherical unit vector relations

Spherical in Cartesian.

$$\hat{e}_r = \sin\theta\cos\phi\,\hat{e}_x + \sin\theta\sin\phi\,\hat{e}_y + \cos\theta\,\hat{e}_z$$
$$\hat{e}_\theta = \cos\theta\cos\phi\,\hat{e}_x + \cos\theta\sin\phi\,\hat{e}_y - \sin\theta\,\hat{e}_z$$
$$\hat{e}_\phi = -\sin\phi\,\hat{e}_x + \cos\phi\,\hat{e}_y$$

Inverse.

$$\hat{e}_x = \sin\theta\cos\phi\,\hat{e}_r + \cos\theta\cos\phi\,\hat{e}_\theta - \sin\phi\,\hat{e}_\phi$$
$$\hat{e}_y = \sin\theta\sin\phi\,\hat{e}_r + \cos\theta\sin\phi\,\hat{e}_\theta + \cos\phi\,\hat{e}_\phi$$
$$\hat{e}_z = \cos\theta\,\hat{e}_r - \sin\theta\,\hat{e}_\theta$$

Constant vs. position-dependent unit vectors

Note.
In polar, cylindrical, and spherical coordinates, unit vectors change direction with position, unlike Cartesian unit vectors.

Why it matters.
When we differentiate position with respect to time, the unit vectors themselves contribute terms. Cartesian is the only system where $d\hat{e}/dt = \vec{0}$.

3 Velocity and Acceleration in Different Coordinates

General definitions

Velocity.
The time derivative of position vector:

$$\vec{v} = \frac{d\vec{r}}{dt}$$

Acceleration.
The time derivative of velocity vector:

$$\vec{a} = \frac{d\vec{v}}{dt} = \frac{d^2\vec{r}}{dt^2}$$

Cartesian coordinates

Position.

$\vec{r} = x\hat{e}_x + y\hat{e}_y + z\hat{e}_z$.

Velocity.

$$\vec{v} = \dot{x}\hat{e}_x + \dot{y}\hat{e}_y + \dot{z}\hat{e}_z = v_x\hat{e}_x + v_y\hat{e}_y + v_z\hat{e}_z$$

where $v_x = \dot{x}$, $v_y = \dot{y}$, $v_z = \dot{z}$.

Acceleration.

$$\vec{a} = \ddot{x}\hat{e}_x + \ddot{y}\hat{e}_y + \ddot{z}\hat{e}_z = a_x\hat{e}_x + a_y\hat{e}_y + a_z\hat{e}_z$$

Note.

In Cartesian coordinates, $d\hat{e}_x/dt = d\hat{e}_y/dt = d\hat{e}_z/dt = \vec{0}$ (unit vectors are constant).

Polar coordinates: position and velocity

Position.
$\vec{r} = r\hat{e}_r$ (no $\hat{e}_\theta$ component for position).

Velocity setup.

$$\vec{v} = \frac{d\vec{r}}{dt} = \frac{dr}{dt}\hat{e}_r + r\frac{d\hat{e}_r}{dt}$$

Since $\hat{e}_r = (\cos\theta, \sin\theta)$, we have $\dfrac{d\hat{e}_r}{dt} = \hat{e}_\theta \cdot \dfrac{d\theta}{dt}$.

Velocity.

$$\vec{v} = \dot{r}\hat{e}_r + r\dot{\theta}\hat{e}_\theta = v_r\hat{e}_r + v_\theta\hat{e}_\theta$$

where $v_r = \dot{r}$ and $v_\theta = r\dot{\theta}$.

Polar coordinates: acceleration

Differentiate velocity.

$$\vec{a} = (\ddot{r}\hat{e}_r + \dot{r}\dot{\hat{e}}_r) + (\dot{r}\dot{\theta}\hat{e}_\theta + r\ddot{\theta}\hat{e}_\theta + r\dot{\theta}\dot{\hat{e}}_\theta)$$

Using $\dfrac{d\hat{e}_\theta}{dt} = -\hat{e}_r \cdot \dfrac{d\theta}{dt}$:

$$\vec{a} = (\ddot{r} - r\dot{\theta}^2)\hat{e}_r + (r\ddot{\theta} + 2\dot{r}\dot{\theta})\hat{e}_\theta = a_r\hat{e}_r + a_\theta\hat{e}_\theta$$

where $a_r = \ddot{r} - r\dot{\theta}^2$ and $a_\theta = r\ddot{\theta} + 2\dot{r}\dot{\theta}$.

Key derivatives.

$\dfrac{d\hat{e}_r}{dt} = \dot{\theta}\hat{e}_\theta, \quad \dfrac{d\hat{e}_\theta}{dt} = -\dot{\theta}\hat{e}_r.$

Polar: important note on $\hat{e}_\theta$

$\hat{e}_\theta$ is a unit vector.
Magnitude 1 (dimensionless), NOT along the circular arc. It points in the direction perpendicular to $\hat{e}_r$, not along the arc length.

Angles and rates.
θ has units of rad (dimensionless in SI). $\dot{\theta}$ has units of rad/s (angular velocity).

Tangential velocity.
The tangential velocity component $v_\theta = r\dot{\theta}$ has units of m/s (linear velocity).

Normal and tangential coordinates

Setup.
For a particle moving along a curved path, we define unit vectors $\hat{e}_t$ (tangential) and $\hat{e}_n$ (normal).

Position and velocity.
$\vec{r}$ runs along the path; velocity is

$$\vec{v} = v\hat{e}_t$$

where $v = |\vec{v}|$ is the speed.

Note.
Velocity has no normal component (always tangent to path).

Normal-tangential: acceleration

Rotation of $\hat{e}_t$.
Since $\hat{e}_t = (\cos\beta, \sin\beta)$, the unit vector rotates as the path turns:

$$\frac{d\hat{e}_t}{dt} = (-\sin\beta, \cos\beta) \cdot \frac{d\beta}{dt} = \hat{e}_n \cdot \frac{d\beta}{dt}$$

Acceleration.
Using $\dfrac{d\beta}{dt} = \dfrac{v}{\rho}$ where ρ is the radius of curvature:

$$\vec{a} = \frac{dv}{dt}\hat{e}_t + \frac{v^2}{\rho}\hat{e}_n = a_t\hat{e}_t + a_n\hat{e}_n$$

$a_t = dv/dt$ changes speed. $a_n = v^2/\rho$ is centripetal and only changes direction, not speed.

Cylindrical coordinates: velocity

Position.

$\vec{r} = \rho\hat{e}_\rho + z\hat{e}_z$.

Velocity setup.

$$\vec{v} = \frac{d\vec{r}}{dt} = \dot{\rho}\hat{e}_\rho + \rho\frac{d\hat{e}_\rho}{dt} + \dot{z}\hat{e}_z$$

Since $\hat{e}_\rho = (\cos\phi, \sin\phi, 0)$, $\dfrac{d\hat{e}_\rho}{dt} = \dot{\phi}\hat{e}_\phi$.

Velocity.

$$\vec{v} = \dot{\rho}\hat{e}_\rho + \rho\dot{\phi}\hat{e}_\phi + \dot{z}\hat{e}_z = v_\rho\hat{e}_\rho + v_\phi\hat{e}_\phi + v_z\hat{e}_z$$

where $v_\rho = \dot{\rho}$, $v_\phi = \rho\dot{\phi}$, $v_z = \dot{z}$.

Cylindrical coordinates: acceleration

Differentiate velocity.
Using $\frac{d\hat{e}_\rho}{dt} = \dot{\phi}\hat{e}_\phi$ and $\frac{d\hat{e}_\phi}{dt} = -\dot{\phi}\hat{e}_\rho$:

$$\vec{a} = (\ddot{\rho} - \rho\dot{\phi}^2)\hat{e}_\rho + (\rho\ddot{\phi} + 2\dot{\rho}\dot{\phi})\hat{e}_\phi + \ddot{z}\hat{e}_z$$

where $a_\rho = \ddot{\rho} - \rho\dot{\phi}^2$, $a_\phi = \rho\ddot{\phi} + 2\dot{\rho}\dot{\phi}$, $a_z = \ddot{z}$.

Key derivatives.
$\frac{d\hat{e}_\rho}{dt} = \dot{\phi}\hat{e}_\phi, \quad \frac{d\hat{e}_\phi}{dt} = -\dot{\phi}\hat{e}_\rho, \quad \frac{d\hat{e}_z}{dt} = \vec{0}.$

4 Physics of the Current Simulator

Vocabulary first, simulator next

Where the math runs.
The Rust crate `haircut_core` is where vectors, normals, and time derivatives become running code.

Reading style.
Each slide pairs a short snippet from the crate with the mathematical statement it implements. The point is to recognise the formula behind the line of code.

Scope.
A rigid head, a kinematic clipper, point-mass hair strands, and free-falling debris. No haptics, no friction, no per-strand collision. The omissions are intentional.

Vec3: dot product and length

```
pub fn dot(self, other: Self) -> f32 {
    self.x * other.x + self.y * other.y + self.z * other.z
}
pub fn length(self) -> f32 { self.length_squared().sqrt() }
```

Mathematical meaning.

The Cartesian formulas from earlier: $\vec{A} \cdot \vec{B} = A_x B_x + A_y B_y + A_z B_z$ and $|\vec{A}| = \sqrt{\vec{A} \cdot \vec{A}}$. The struct holds three `f32` fields, so a `Vec3` is exactly the triple (A_x, A_y, A_z).

Safe normalisation

```
pub fn normalized(self) -> Self {
    let len = self.length();
    if len <= 1.0e-6 { Self::ZERO } else { self / len }
}
```

Mathematical meaning.
$\hat{e}_A = \vec{A}/|\vec{A}|$, except the division is undefined when $|\vec{A}| = 0$. We map the degenerate case to $\vec{0}$ rather than producing `NaN`, so downstream contact and normal code stays finite.

Head as an ellipsoid: implicit form

```
pub fn normalized_distance_squared(&self, p: Vec3) -> f32 {
    let rel = p - self.center;
    (rel.x / rx).powi(2) + (rel.y / ry).powi(2) + (rel.z / rz).powi(2)
}
```

Mathematical meaning.
The scalp is the level set $f(\vec{p}) = 1$ of

$$f(x, y, z) = \left(\frac{x - c_x}{r_x}\right)^2 + \left(\frac{y - c_y}{r_y}\right)^2 + \left(\frac{z - c_z}{r_z}\right)^2.$$

$f < 1$ is inside the head, $f = 1$ is the surface, $f > 1$ is outside.

Surface normal from the gradient

```
let gradient = Vec3::new(
    rel.x / rx.powi(2),
    rel.y / ry.powi(2),
    rel.z / rz.powi(2),
);
gradient.normalized()
```

Mathematical meaning.
For an implicit surface $f(\vec{p}) = 1$ the outward normal is parallel to ∇f. Here $\nabla f = 2\left(\frac{x-c_x}{r_x^2}, \frac{y-c_y}{r_y^2}, \frac{z-c_z}{r_z^2}\right)$; the factor of 2 disappears after we normalise.

Projection to the surface

```
let rel = p - self.center;
let norm_sq = self.normalized_distance_squared(p);
let factor = 1.0 / norm_sq.sqrt();
self.center + rel * factor
```

Mathematical meaning.
Take the ray from the head centre through $\vec{p}$, then rescale so $f = 1$. With $s = 1/\sqrt{f(\vec{p})}$, the projection is $\vec{c} + s(\vec{p} - \vec{c})$. Exact for a sphere, only approximate for an ellipsoid (it is not the true closest-point projection).

Resolving clipper-head contact

```
if self.contains(sphere_center) {
    let surface = self.project_to_surface(sphere_center);
    let normal = self.surface_normal(surface);
    surface + normal * sphere_radius.max(0.0)
} else { sphere_center }
```

Mathematical meaning.
If the clipper sphere centre is inside the head, push it out to the surface and then offset by the radius along the outward normal:

$$\vec{p}_{\text{out}} = \vec{p}_{\text{surf}} + R\,\hat{n}(\vec{p}_{\text{surf}}).$$

This drives the signed distance from clipper sphere to scalp to zero.

A strand is a chain of nodes

```
pub struct HairRoot { pub offset: Vec3, pub normal: Vec3 }
pub struct HairNode { pub position: Vec3, pub velocity: Vec3 }
pub struct HairStrand {
    pub root: HairRoot,
    pub nodes: Vec<HairNode>,
    pub active_len: usize,
}
```

Mathematical meaning.
A strand is a sequence of N point masses $\{\vec{p}_0, \vec{p}_1, \ldots, \vec{p}_{N-1}\}$ pinned at the root, each with its own velocity $\vec{v}_i$. The target segment length is $L_{\text{seg}} = L_{\text{hair}}/(N-1)$.

Initial geometry along the normal

```
let direction = self.root.normal.normalized();
for (index, node) in self.nodes.iter_mut().enumerate() {
    let dist = segment_length * index as f32;
    node.position = root_position + direction * dist;
    node.velocity = Vec3::ZERO;
}
```

Mathematical meaning.
At rest, $\vec{p}_i = \vec{p}_{\text{root}} + i\, L_{\text{seg}}\, \hat{n}$, with $\hat{n}$ the outward scalp normal at the root. The strand stands straight along the radial direction; all velocities start at zero.

Per-node forces

```
let stiffness = rigidity * (1.0 - index as f32 / stiffness_denom);
node.velocity += gravity * dt;
node.velocity += normal * (stiffness * dt);
```

Mathematical meaning.
Two accelerations: uniform gravity $\vec{g}$, plus a position-dependent rest-direction stiffness $k_i\,\hat{n}$ with $k_i = k_0\,(1 - i/(N-1))$ decaying along the strand. Each contributes its impulse $\vec{a}\,\Delta t$ to the velocity.

Time integration: symplectic Euler

```
node.velocity += gravity * dt;
node.velocity += normal * (stiffness * dt);
node.position += node.velocity * dt;
node.velocity *= damping;
```

Mathematical meaning.
Semi-implicit (symplectic) Euler: update velocity first, then advance position with the new velocity,

$$\vec{v}_{k+1} = \vec{v}_k + \vec{a}\,\Delta t, \qquad \vec{p}_{k+1} = \vec{p}_k + \vec{v}_{k+1}\,\Delta t.$$

Then $\vec{v} \leftarrow d\,\vec{v}$ with $d < 1$ approximates linear viscous drag.

Segment-length constraint

```
let curr_vec = node.position - prev_pos;
let curr_len = curr_vec.length();
if curr_len > 1.0e-6 {
    let correction = ((curr_len - target_segment_length) / curr_len) * 0.85;
    node.position -= curr_vec * correction;
}
```

Mathematical meaning.
Position-based correction (PBD-style). If $\ell_i = |\vec{p}_i - \vec{p}_{i-1}| \neq L_{\text{seg}}$, shift $\vec{p}_i$ along the segment to remove $0.85\,(\ell_i - L_{\text{seg}})$ of the excess each step. Stable for stiff constraints without solving Hooke's law.

Cut detection

```
let cut_radius_sq = (clipper.radius * 0.95).powi(2);
if clipper.is_cutting
    && node.position.distance_squared(clipper.actual_pos) < cut_radius_sq
{
    cut_idx = Some(index);
    break;
}
```

Mathematical meaning.
A node is cut if it lies inside a sphere of radius $0.95\,R$ around the clipper:

$$|\vec{p}_i - \vec{p}_{\text{clip}}|^2 < (0.95\,R)^2.$$

Using squared distance avoids a square root; the predicate is unchanged.

Cut update: detach the tail

```
let tail_points = strand.nodes[cut_idx..active_len]
    .iter()
    .map(|node| DebrisPoint { position: node.position, velocity: ..., is_stopped: false })
    .collect();
debris_out.push(DebrisSegment::new(tail_points));
strand.shorten_to(cut_idx);
```

Mathematical meaning.
At the cut index k, partition the strand: $\{\vec{p}_0, \ldots, \vec{p}_{k-1}\}$ stays attached, and $\{\vec{p}_k, \ldots, \vec{p}_{N-1}\}$ becomes a free debris segment with a small randomised initial velocity.

Debris dynamics

```
point.velocity += gravity * (1.5 * dt);
point.position += point.velocity * dt;
if point.position.z <= floor_z {
    point.position.z = floor_z;
    point.velocity = Vec3::ZERO;
    point.is_stopped = true;
}
```

Mathematical meaning.
Each debris point is a free particle under augmented gravity $1.5\,\vec{g}$ (visual snappiness), integrated with symplectic Euler. On contact with the floor plane $z = z_{\text{floor}}$, the point snaps to the plane, loses its velocity, and freezes.

What is intentionally deferred

Not modelled.
Per-strand collisions, hair-hair friction, bending stiffness, contact forces between blade and hair, deformable scalp, thermal effects, hair-blade haptics.

Why.
The gating layer needs a deterministic, replayable physics envelope, not a research-grade simulation. Every omission keeps the per-frame work small and the audit log reproducible.

5 Easy JavaScript 3D Renderers

Why a renderer matters here

The simulator is headless.
`haircut_core` produces positions and normals each step. A renderer is what turns those numbers into something a user can see and trust.

Three options worth knowing.
Three.js gives you the most control, Babylon.js bundles a full game engine, and A-Frame lets you describe a scene in HTML tags. All three run in any modern browser.

Pick by friction, not by hype.
For a course demo, the right framework is the one you can stand up in ten minutes.

Three.js: minimal usage

Demo file.
`Code_Week7/threejs_demo.html` – self-contained, opens in any modern browser, library pulled from a CDN.

Shape of a Three.js page.
Import the module, create a `Scene`, a `PerspectiveCamera`, and a `WebGLRenderer`; add meshes (geometry + material) to the scene; call `setAnimationLoop` to render each frame.

What the demo renders.
An ellipsoid "head" with ~ 200 short line segments along the outward ellipsoid normal – the same gradient-normal formula from the simulator section.

Three.js: features

What you get.
A thin WebGL wrapper: `Scene`, `Camera`, `Mesh`, `Material`, `Light`. No engine, no scene editor, no opinions on physics.

When it fits.
You want exact control over geometry and shaders, you are happy to glue your own loop, and you do not need built-in physics or audio.

Trade-offs.
Tiny core, huge ecosystem of loaders and helpers, but you write more boilerplate than the other two for anything beyond a hello-world scene.

Babylon.js: minimal usage

Demo file.
`Code_Week7/babylonjs_demo.html` – self-contained, mouse drag orbits the head out of the box.

Shape of a Babylon page.
Bind an `Engine` to a canvas, build a `Scene`, attach an `ArcRotateCamera` and a light, instantiate meshes via `MeshBuilder`, then call `runRenderLoop`.

What the demo renders.
Same head + hair-normal scene as the Three.js demo, but with Babylon's built-in orbit camera and a hemispheric light – almost no boilerplate beyond the scene set-up.

Babylon.js: features

What you get.
A full game engine: scene graph, physics plugins, post-processing, an asset pipeline, a built-in inspector, GUI widgets, and an arc-rotate camera that already orbits the head with the mouse.

When it fits.
You want a complete authoring stack and you are willing to ship a larger runtime; demos with light, materials, and camera controls work out of the box.

Trade-offs.
More batteries included than Three.js, with the bundle size and API surface to match.

A-Frame: minimal usage

Demo file.
`Code_Week7/aframe_demo.html` – a full scene with no JavaScript at all; everything is HTML tags.

Shape of an A-Frame page.
Load the library, then nest custom tags inside `<a-scene>`: each primitive (sphere, cylinder, sky, camera) is a tag with attributes for position, scale, and material.

Mental model.
A scene is HTML. Every primitive is a tag, every property is an attribute – a teaching assistant can edit the geometry without touching JavaScript.

A-Frame: features

What you get.
A declarative wrapper around Three.js with an entity-component system, default WebXR support, a built-in scene inspector (Ctrl+Alt+I), and ready-made primitives.

When it fits.
You want to publish a scene a teaching assistant can edit in HTML, or you want VR/AR on day one without learning the WebXR API.

Trade-offs.
Easiest entry, but anything custom drops you back into Three.js underneath. Best as a presentation layer, not a final engine.

Try them now: the three demos

Files in `Code_Week7/`.
`threejs_demo.html`, `babylonjs_demo.html`, `aframe_demo.html` – one self-contained page per framework, each rendering an ellipsoid head with hair-like geometry along the outward normal.

How to run.
Open any of the three files in a modern browser. No build step, no server, no package install – each library is pulled from its CDN by the page itself.

Why three.
Reading the three versions side by side is the fastest way to feel the trade-offs: imperative core, batteries-included engine, or pure-HTML scene.

Takeaway

Vectors are the language of 3D.
Magnitude plus direction; addition, dot, and cross are the core operations.

Pick the coordinate system that matches the geometry.
Cartesian for grids and world frames, cylindrical for axis-aligned reach, spherical for radial sweeps.

Curvilinear unit vectors move with position.
That is why polar, cylindrical, and spherical accelerations carry the extra terms $-r\dot{\theta}^2$ and $2\dot{r}\dot{\theta}$.

Choice of system changes the math, not the physics.
The trajectory is the same; the algebra is simpler in the right frame.